COURS D'ALGORITHME AVANCE (SUITE)

CHAP 4. LES ARBRES

Introduction

Un organisateur de tournoi de rugby recherche la meilleure solution pour afficher les potentiels quarts de final, demi-finales et finale :

Au départ nous avons 4 poules de 4 équipes. Les 4 équipes d'une poule s'affrontent dans un mini championnat (3 matchs par équipe). À l'issue de cette phase de poule, les 2 premières équipes de chaque poule sont qualifiées pour les quarts de finale.

Dans ce qui suit, on désigne les 2 qualifiés par poule par :

- Poule 1 => 1er Eq1 ; 2e Eq8
- Poule 2 => 1er Eq2 ; 2e Eq7
- Poule 3 => 1er Eq3 ; 2e Eq6
- Poule 4 => 1er Eq4 ; 2e Eq5

En quart de final on va avoir :

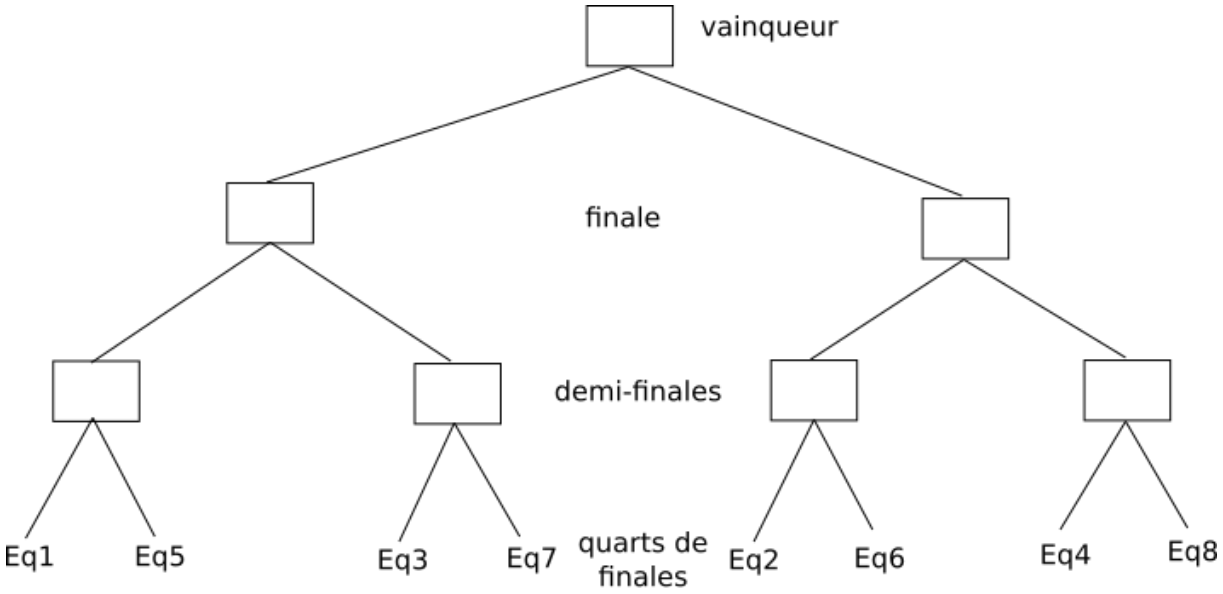
- quart de finale 1 => Eq1 contre Eq5
- quart de finale 2 => Eq2 contre Eq6
- quart de finale 3 => Eq3 contre Eq7
- quart de finale 4 => Eq4 contre Eq8

Pour les demi-finales on aura :

- demi-final 1 => vainqueur quart de finale 1 contre vainqueur quart de finale 3
- demi-final 2 => vainqueur quart de finale 2 contre vainqueur quart de finale 4

L'organisateur du tournoi affiche les informations ci-dessus le jour du tournoi. Malheureusement, la plupart des spectateurs se perdent quand ils cherchent à déterminer les potentielles demi-finales (et ne parlons pas de la finale !)

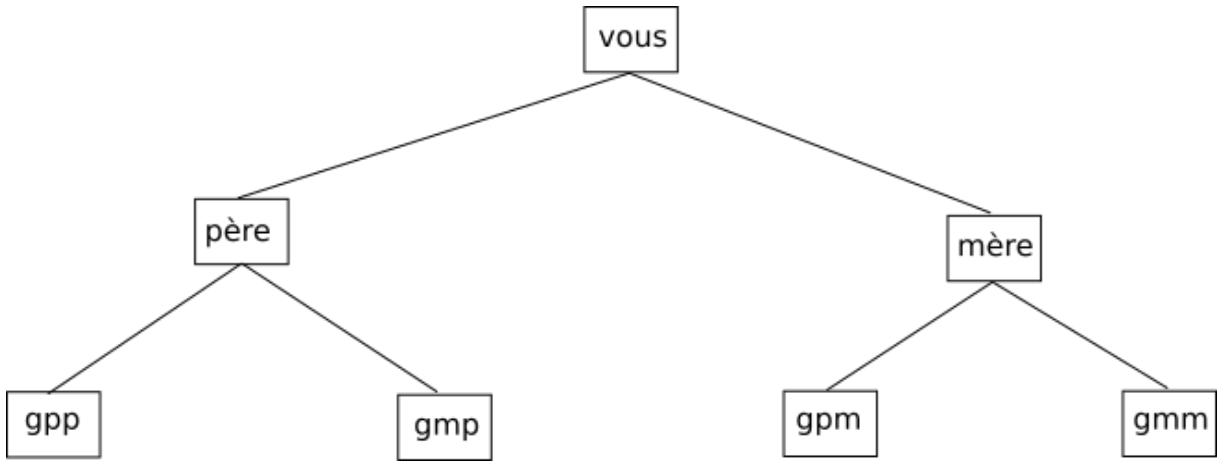
Pourtant, un simple graphique aurait grandement simplifié les choses :



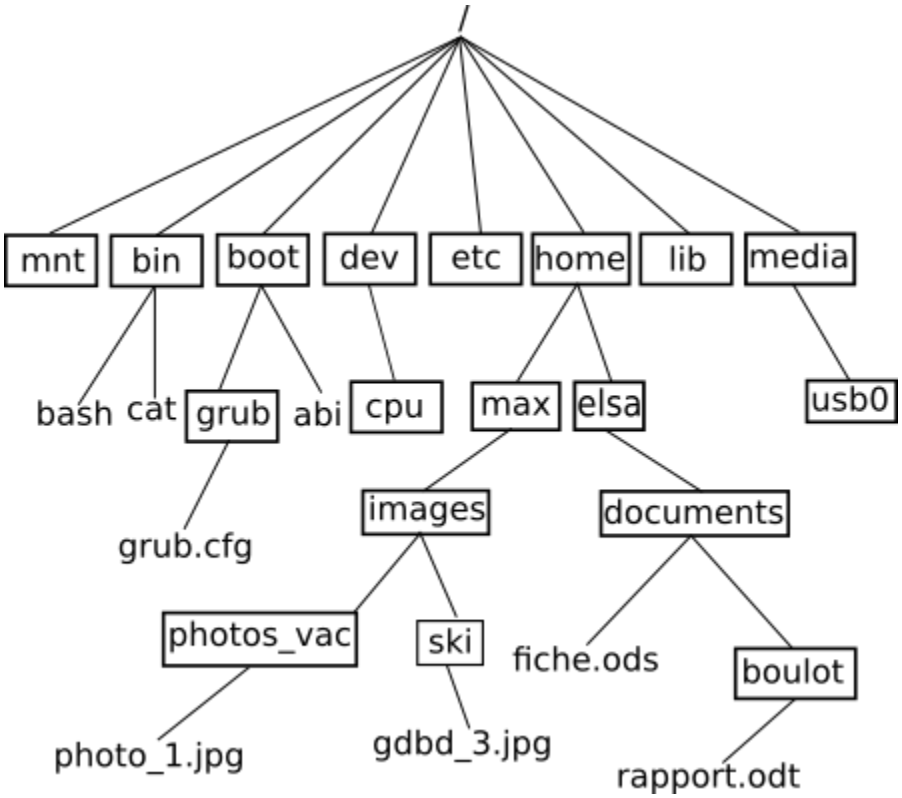
Poule 1 => 1er Eq1 ; 2e Eq8
Poule 2 => 1er Eq2 ; 2e Eq7
Poule 3 => 1er Eq3 ; 2e Eq6
Poule 4 => 1er Eq4 ; 2e Eq5

Les spectateurs peuvent alors recopier sur un bout de papier ce schéma et ensuite se livrer au jeu des pronostiques.

Nous avons ci-dessous ce que l'on appelle une structure en arbre. On peut aussi retrouver cette même structure dans un arbre généalogique :



Dernier exemple, les systèmes de fichiers dans les systèmes de type UNIX ont aussi une structure en arbre

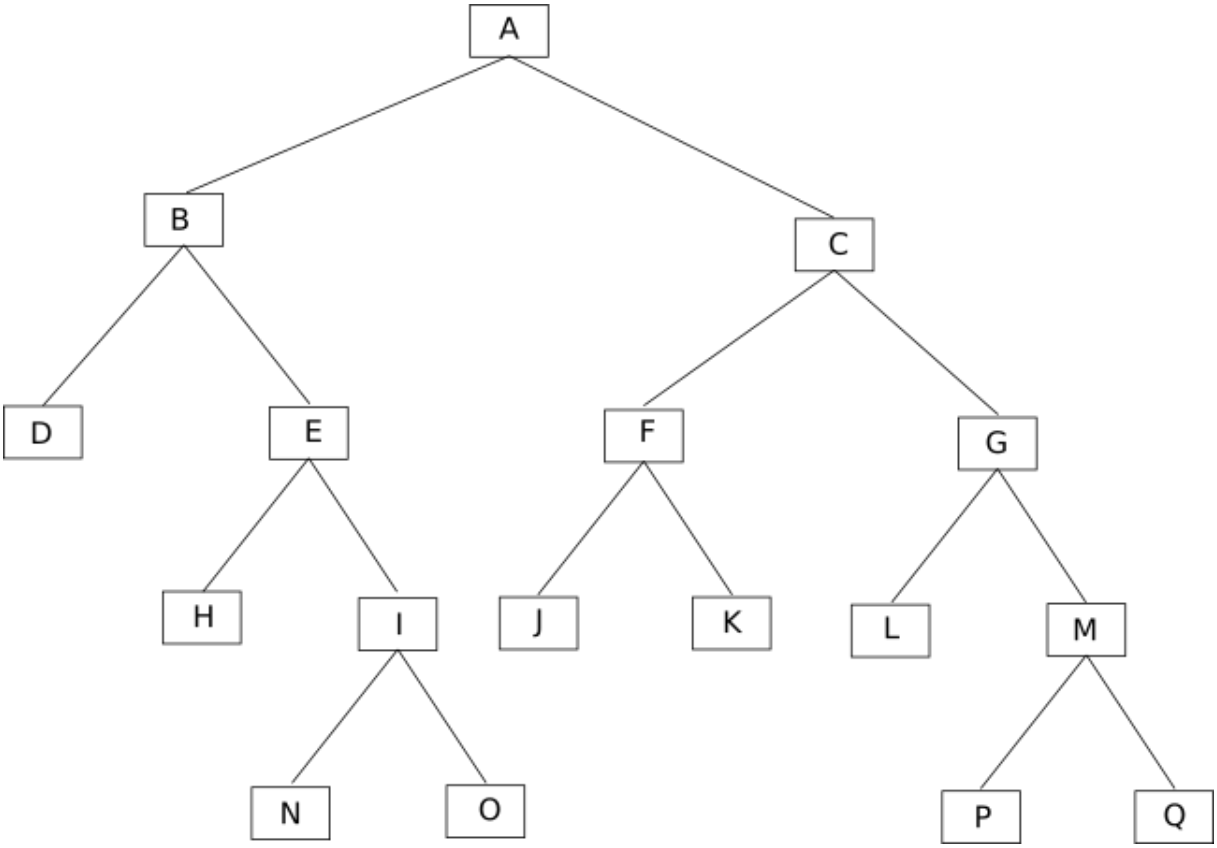


système de fichiers de type UNIX

Les arbres sont des types abstraits très utilisés en informatique. On les utilise notamment quand on a besoin d'une structure hiérarchique des données : dans l'exemple ci-dessous le fichier grub.cfg ne se trouve pas au même niveau que le fichier rapport.odt (le fichier grub.cfg se trouve "plus proche" de la racine / que le fichier rapport.odt). On ne pourrait pas avec une simple liste qui contiendrait les noms des fichiers et des répertoires, rendre compte de cette hiérarchie (plus ou moins "proche" de la racine). On trouve souvent dans cette hiérarchie une notion de temps (les quarts de finale sont avant les demi-finales ou encore votre grand-mère paternelle est née avant votre père), mais ce n'est pas une obligation (voir l'arborescence du système de fichiers).

Les arbres binaires sont des cas particuliers d'arbre : l'arbre du tournoi de rugby et l'arbre "père, mère..." sont des arbres binaires, en revanche, l'arbre représentant la structure du système de fichier n'est pas un arbre binaire. Dans un arbre binaire, on a au maximum 2 branches qui partent d'un élément (pour le système de fichiers, on a 7 branches qui partent de la racine : ce n'est donc pas un arbre binaire). Dans la suite nous allons uniquement travailler sur les arbres binaires.

Soit l'arbre binaire suivant :



Un peu de vocabulaire :

- chaque élément de l'arbre est appelé noeud (par exemple : A, B, C, D,...,P et Q sont des noeuds)
- le noeud initial (A) est appelé noeud racine ou plus simplement racine
- On dira que le noeud E et le noeud D sont les fils du noeud B. On dira que le noeud B est le père des noeuds E et D
- Dans un arbre binaire, un noeud possède au plus 2 fils
- Un noeud n'ayant aucun fils est appelé feuille (exemples : D, H, N, O, J, K, L, P et Q sont des feuilles)
- À partir d'un noeud (qui n'est pas une feuille), on peut définir un sous-arbre gauche et un sous-arbre droite (exemple : à partir de C on va trouver un sous-arbre gauche composé des noeuds F, J et K et un sous-arbre droit composé des noeuds G, L, M, P et Q)
- On appelle arête le segment qui relie 2 noeuds.
- On appelle taille d'un arbre le nombre de noeuds présents dans cet arbre

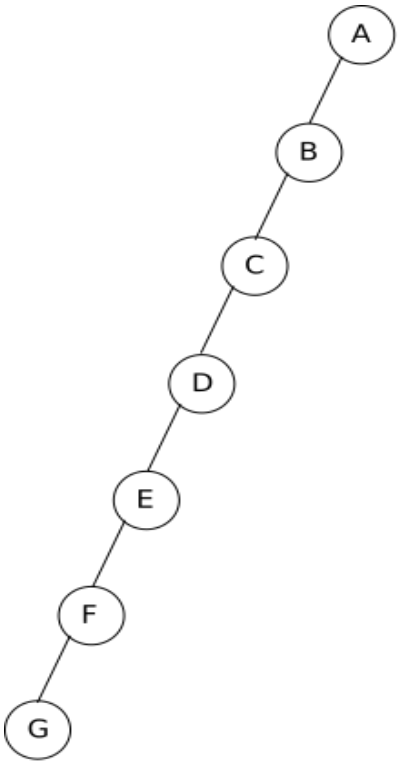
- On appelle profondeur d'un nœud ou d'une feuille dans un arbre binaire le nombre de nœuds du chemin qui va de la racine à ce nœud. La racine d'un arbre est à une profondeur 1, et la profondeur d'un nœud est égale à la profondeur de son prédécesseur plus 1. Si un noeud est à une profondeur p, tous ses successeurs sont à une profondeur p+1. Exemples : profondeur de B = 2 ; profondeur de I = 4 ; profondeur de P = 5 **ATTENTION** : on trouve aussi dans certains livres la profondeur de la racine égale à 0 (on trouve alors : profondeur de B = 1 ; profondeur de I = 3 ; profondeur de P = 4). Les 2 définitions sont valables, il faut juste préciser si vous considérez que la profondeur de la racine est de 1 ou de 0.
- On appelle hauteur d'un arbre la profondeur maximale des nœuds de l'arbre. Exemple : la profondeur de P = 5, c'est un des noeuds les plus profond, donc la hauteur de l'arbre est de 5. **ATTENTION** : comme on trouve 2 définitions pour la profondeur, on peut trouver 2 résultats différents pour la hauteur : si on considère la profondeur de la racine égale à 1, on aura bien une hauteur de 5, mais si l'on considère que la profondeur de la racine est de 0, on aura alors une hauteur de 4.

Il est aussi important de bien noter que l'on peut aussi voir les arbres comme des structures récursives : les fils d'un noeud sont des arbres (sous-arbre gauche et un sous-arbre droite dans le cas d'un arbre binaire), ces arbres sont eux mêmes constitués d'arbres...

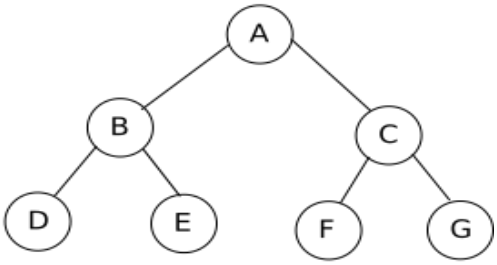
À faire vous-même 1

Trouvez un autre exemple de données qui peuvent être représentées par un arbre binaire (dans le domaine de votre choix). Dessinez au moins une partie de cet arbre binaire. Déterminez la hauteur et la taille de l'arbre que vous aurez dessiné.

Il est possible d'avoir des arbres binaires de même taille mais de "forme" très différente :



Arbre 1



Arbre 2

Sur le schéma ci-dessus l'arbre 1 est dit " filiforme" alors que l'arbre 2 est dit "complet" (on dira qu'un arbre binaire est complet si tous les noeuds possèdent 2 fils et que toutes les feuilles se situent à la même profondeur). On pourra aussi dire que l'arbre 1 est déséquilibré alors que l'arbre 2 est équilibré.

Si on prend un arbre filiforme de taille n, on peut dire que la hauteur de cet arbre est égale à $n-1$ (si on prend la définition de la hauteur d'un arbre où la racine a une profondeur 0)
Si on prend un arbre complet de taille **n**, on peut démontrer que la hauteur de cet arbre est égale à la partie entière de $\log2(n)$ (on arrondit à l'entier immédiatement inférieur le $\log2(n)$). Dans le cas de l'arbre 2, nous avons $\log2(7)=2{,}8$ donc en prenant la partie entière on a bien la hauteur de l'arbre 2 égale à 2.
Un arbre filiforme et un arbre complet étant deux cas extrêmes, on peut affirmer que pour un arbre binaire quelconque :

$$\lfloor \log2(n) \rfloor \leq h \leq n-1$$

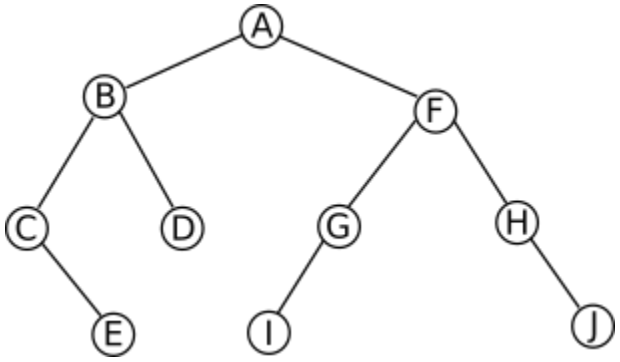
avec n la taille de l'arbre et h la hauteur de l'arbre ($\lfloor \log2(n) \rfloor$ permet de prendre la partie entière du logarithme base 2 de n)

Algorithme sur les arbres binaires

Avant d'entrer dans le vif du sujet (les algorithmes), nous allons un peu approfondir la notion d'arbre binaire :

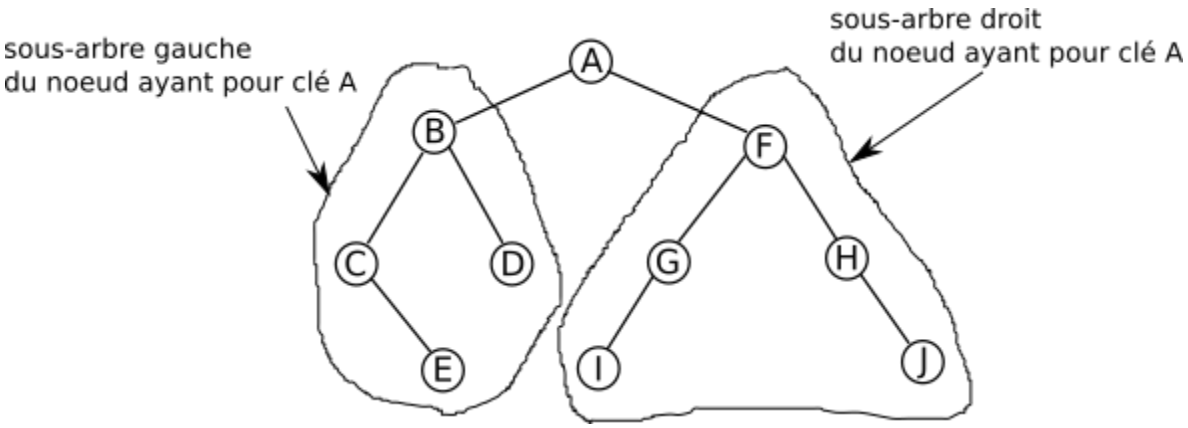
À chaque noeud d'un arbre binaire, on associe une clé ("valeur" associée au noeud on peut aussi utiliser le terme "valeur" à la place de clé), un "sous-arbre gauche" et un "sous-arbre droit"

Soit l'arbre binaire suivant :



si on prend le noeud ayant pour clé A (le noeud racine de l'arbre) on a :

- le sous-arbre gauche est composé du noeud ayant pour clé B, du noeud ayant pour clé C, du noeud ayant pour clé D et du noeud ayant pour clé E
- le sous-arbre droit est composé du noeud ayant pour clé F, du noeud ayant pour clé G, du noeud ayant pour clé H, du noeud ayant pour clé I et du noeud ayant pour clé J



si on prend le noeud ayant pour clé B on a :

- le sous-arbre gauche est composé du noeud ayant pour clé C et du noeud ayant pour clé E
- le sous-arbre droit est uniquement composé du noeud ayant pour clé D

Un arbre (ou un sous-arbre) vide est noté NIL (NIL est une abréviation du latin nihil qui veut dire "rien")

si on prend le noeud ayant pour clé G on a :

- le sous-arbre gauche est uniquement composé du noeud ayant pour clé I
- le sous-arbre droit est vide (NIL)

Il faut bien avoir en tête qu'un sous-arbre (droite ou gauche) est un arbre (même s'il contient un seul noeud ou pas de noeud de tout (NIL)).

Soit un arbre T : T.racine correspond au noeud racine de l'arbre T

Soit un noeud x :

- x.gauche correspond au sous-arbre gauche du noeud x
- x.droit correspond au sous-arbre droit du noeud x
- x.clé correspond à la clé du noeud x

Il faut noter que si le noeud x est une feuille, x.gauche et x.droite sont des arbres vides (NIL)

Calculer la hauteur d'un arbre

Nous allons commencer à travailler sur les algorithmes en nous intéressant à l'algorithme qui permet de calculer la hauteur d'un arbre :

À faire vous-même 1

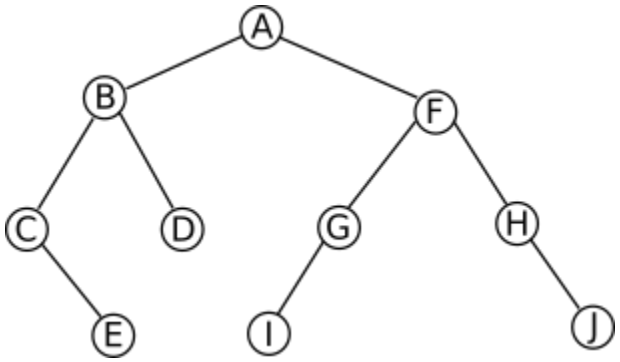
Étudiez cet algorithme :

```
VARIABLE
T : arbre
x : noeud

DEBUT
HAUTEUR(T) :
  si T ≠ NIL :
    x ← T.racine
    renvoyer 1 + max(HAUTEUR(x.gauche), HAUTEUR(x.droit))
  sinon :
    renvoyer 0
  fin si
FIN
```

N.B. la fonction max renvoie la plus grande valeur des 2 valeurs passées en paramètre (exemple : max(5,6) renvoie 6)

Cet algorithme est loin d'être simple, n'hésitez pas à écrire votre raisonnement sur une feuille de brouillon. Vous pourrez par exemple essayer d'appliquer cet algorithme sur l'arbre binaire ci-dessous. N'hésitez pas à poser des questions si nécessaire.



Si vraiment vous avez des difficultés à comprendre le fonctionnement de l'algorithme sur l'arbre ci-dessus, vous trouverez ici un petit calcul qui pourrait vous aider.

Comme vous l'avez sans doute remarqué, nous avons dans l'algorithme ci-dessus une fonction récursive. Vous aurez l'occasion de constater que c'est souvent le cas dans les algorithmes qui travaillent sur des structures de données telles que les arbres.

Calculer la taille d'un arbre

Nous allons maintenant étudier un algorithme qui permet de calculer le nombre de noeuds présents dans un arbre.

À faire vous-même 2

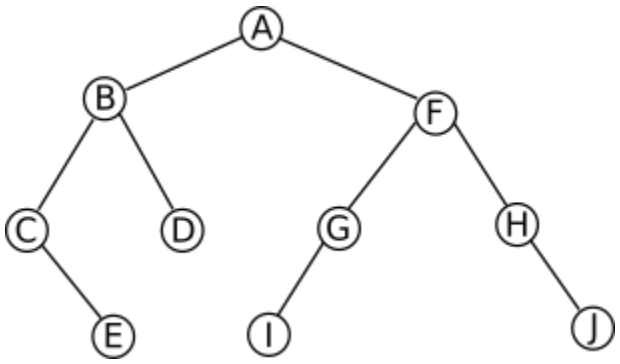
Étudiez cet algorithme :

```
VARIABLE
T : arbre
x : noeud

DEBUT
TAILLE(T) :
  si T ≠ NIL :
    x ← T.racine
    renvoyer 1 + TAILLE(x.gauche) + TAILLE(x.droit)
  sinon :
    renvoyer 0
  fin si
FIN
```

Cet algorithme ressemble beaucoup à l'algorithme étudié au "À faire vous-même 1", son étude ne devrait donc pas vous poser de problème.

Appliquez cet algorithme à l'exemple suivant :



Il existe plusieurs façons de parcourir un arbre (parcourir un arbre = passer par tous les noeuds), nous allons en étudier quelques-unes :

Parcourir un arbre dans l'ordre infixe

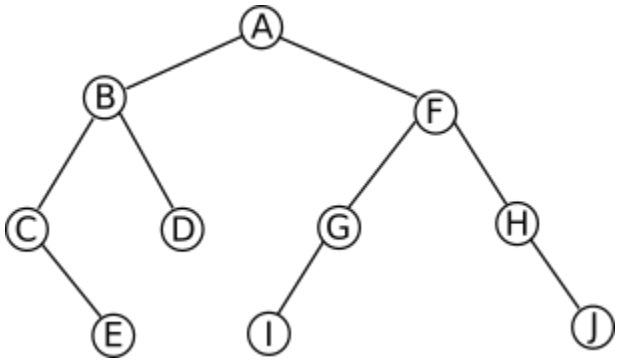
À faire vous-même 3

Étudiez cet algorithme :

```
VARIABLE
T : arbre
x : noeud

DEBUT
PARCOURS-INFIXE(T) :
  si T ≠ NIL :
    x ← T.racine
    PARCOURS-INFIXE(x.gauche)
    affiche x.clé
    PARCOURS-INFIXE(x.droit)
  fin si
FIN
```

Vérifiez qu'en applicant l'algorithme ci-dessus, l'arbre ci-dessous est bien parcouru dans l'ordre suivant : C, E, B, D, A, I, G, F, H, J



Parcourir un arbre dans l'ordre préfixe

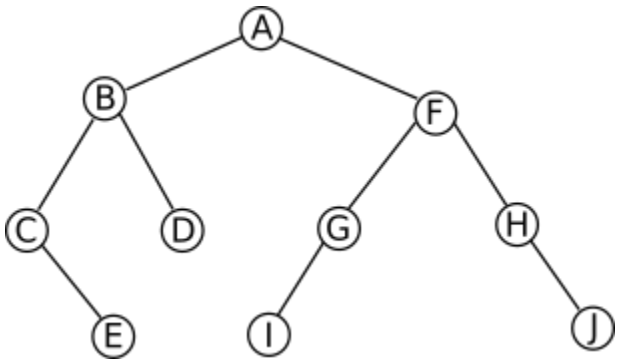
À faire vous-même 4

Étudiez cet algorithme :

```
VARIABLE
T : arbre
x : noeud

DEBUT
PARCOURS-PREFIXE(T) :
  si T ≠ NIL :
    x ← T.racine
    affiche x.clé
    PARCOURS-PREFIXE(x.gauche)
    PARCOURS-PREFIXE(x.droit)
  fin si
FIN
```

Vérifiez qu'en applicant l'algorithme ci-dessus, l'arbre ci-dessous est bien parcouru dans l'ordre suivant : A, B, C, E, D, F, G, I, H, J



Parcourir un arbre dans l'ordre suffixe

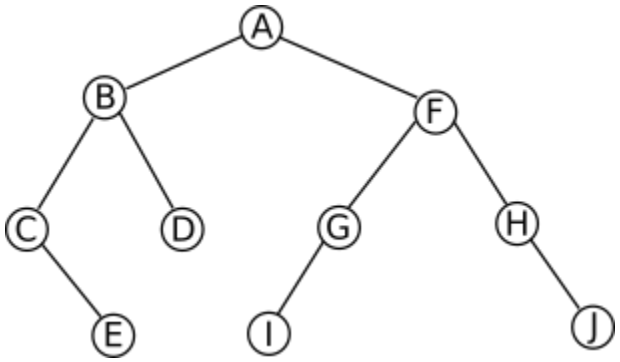
À faire vous-même 5

Étudiez cet algorithme :

```
VARIABLE
T : arbre
x : noeud

DEBUT
PARCOURS-SUFFIXE(T) :
  si T ≠ NIL :
    x ← T.racine
    PARCOURS-SUFFIXE(x.gauche)
    PARCOURS-SUFFIXE(x.droit)
    affiche x.clé
  fin si
FIN
```

Vérifiez qu'en applicant l'algorithme ci-dessus, l'arbre ci-dessous est bien parcouru dans l'ordre suivant : E, C, D, B, I, G, J, H, F, A



Le choix du parcours infixe, préfixe ou suffixe dépend du problème à traiter, on pourra retenir pour les parcours préfixe et suffixe (le cas du parcours infixe sera traité un peu plus loin) que :

- dans le cas du parcours préfixe, un noeud est affiché avant d'aller visiter ces enfants
- dans le cas du parcours suffixe, on affiche chaque nœud après avoir affiché chacun de ses fils

Parcourir un arbre en largeur d'abord

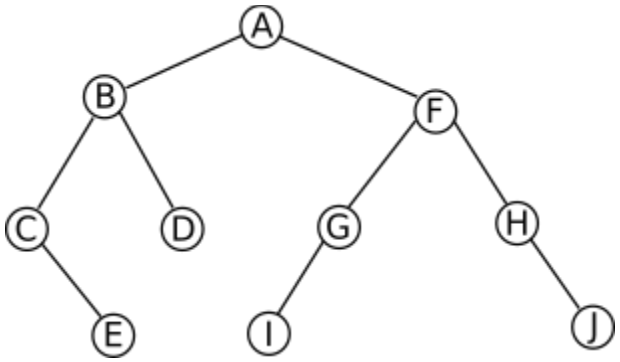
À faire vous-même 6

Étudiez cet algorithme :

```
VARIABLE
T : arbre
Tg : arbre
Td : arbre
x : noeud
f : file (initialement vide)

DEBUT
PARCOURS-LARGEUR(T) :
  enfiler(T.racine, f) //on place la racine dans la file
  tant que f non vide :
    x ← defiler(f)
    affiche x.clé
    si x.gauche ≠ NIL :
      Tg ← x.gauche
      enfiler(Tg.racine, f)
    fin si
    si x.droit ≠ NIL :
      Td ← x.droite
      enfiler(Td.racine, f)
    fin si
  fin tant que
FIN
```

Vérifiez qu'en applicant l'algorithme ci-dessus, l'arbre ci-dessous est bien parcouru dans l'ordre suivant : A, B, F, C, D, G, H, E, I, J



Selon vous, pourquoi parle-t-on de parcours en largeur ?

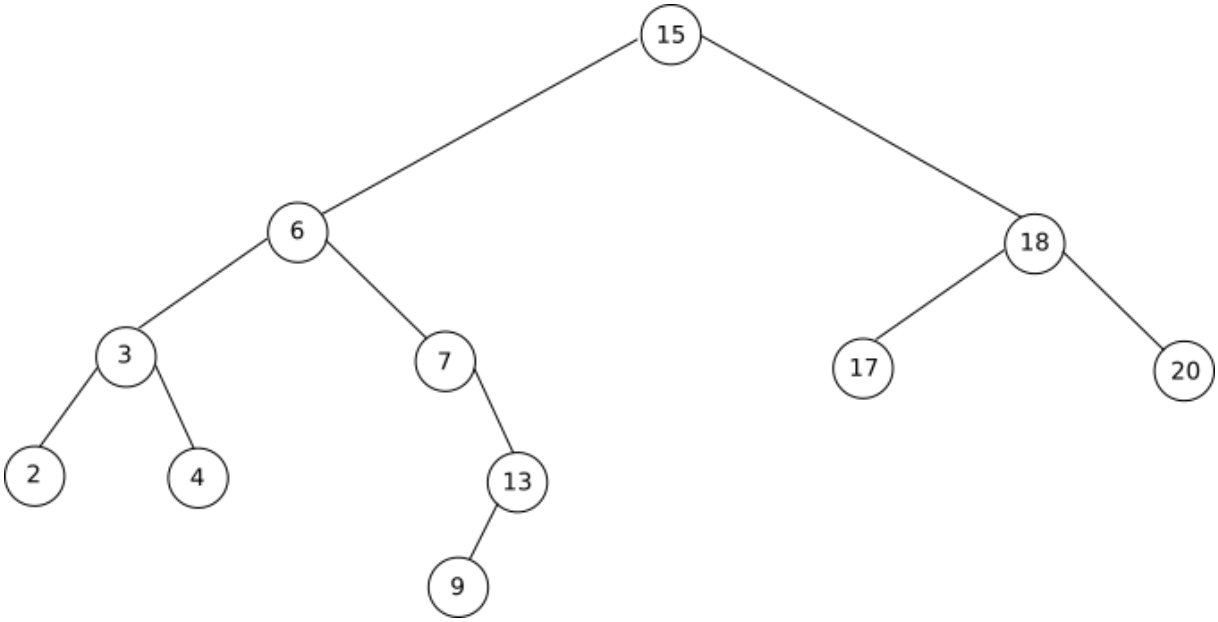
Il est important de bien noter l'utilisation d'une file (FIFO) pour cet algorithme de parcours en largeur. Vous noterez aussi que cet algorithme n'utilise pas de fonction récursive.

Arbre binaire de recherche

Un arbre binaire de recherche est un cas particulier d'arbre binaire. Pour avoir un arbre binaire de recherche :

- il faut avoir un arbre binaire !
- il faut que les clés de noeuds composant l'arbre soient ordonnables (on doit pouvoir classer les noeuds, par exemple, de la plus petite clé à la plus grande)
- soit x un noeud d'un arbre binaire de recherche. Si y est un noeud du sous-arbre gauche de x, alors il faut que y.clé $\leqslant$ x.clé. Si y est un noeud du sous-arbre droit de x, il faut alors que x.clé $\leqslant$ y.clé

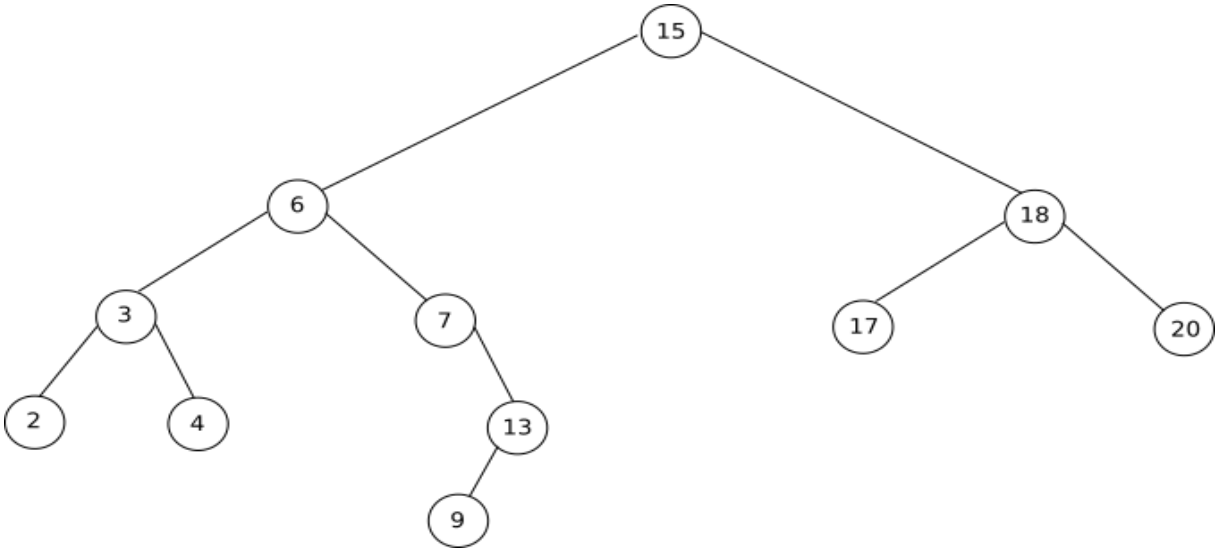
À faire vous-même 7



Vérifiez que l'arbre ci-dessus est bien un arbre binaire de recherche.

À faire vous-même 8

Appliquez l'algorithme de parcours infixe sur l'arbre ci-dessous :



Que remarquez-vous ?

Recherche d'une clé dans un arbre binaire de recherche

Nous allons maintenant étudier un algorithme permettant de rechercher une clé de valeur k dans un arbre binaire de recherche. Si k est bien présent dans l'arbre binaire de recherche, l'algorithme renvoie vrai, dans le cas contraire, il renvoie faux.

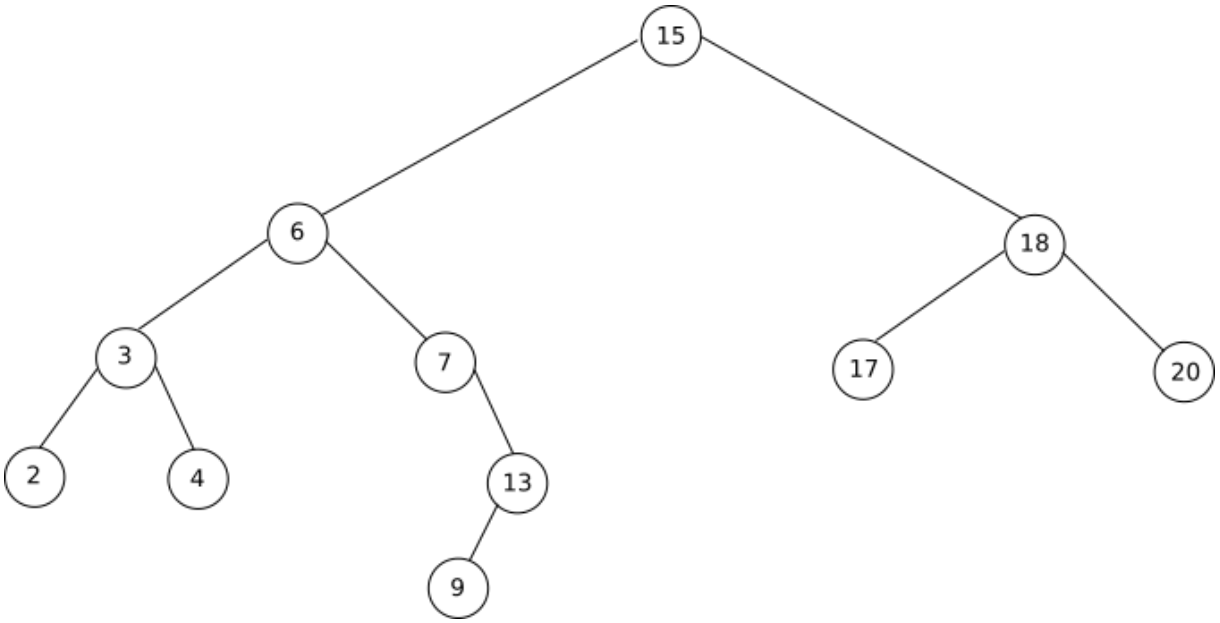
À faire vous-même 9

Étudiez l'algorithme suivant:

```
VARIABLE
T : arbre
x : noeud
k : entier
DEBUT
ARBRE-RECHERCHE(T,k) :
  si T == NIL :
    renvoyer faux
  fin si
  x ← T.racine
  si k == x.clé :
    renvoyer vrai
  fin si
  si k < x.clé :
    renvoyer ARBRE-RECHERCHE(x.gauche,k)
  sinon :
    renvoyer ARBRE-RECHERCHE(x.droit,k)
  fin si
FIN
```

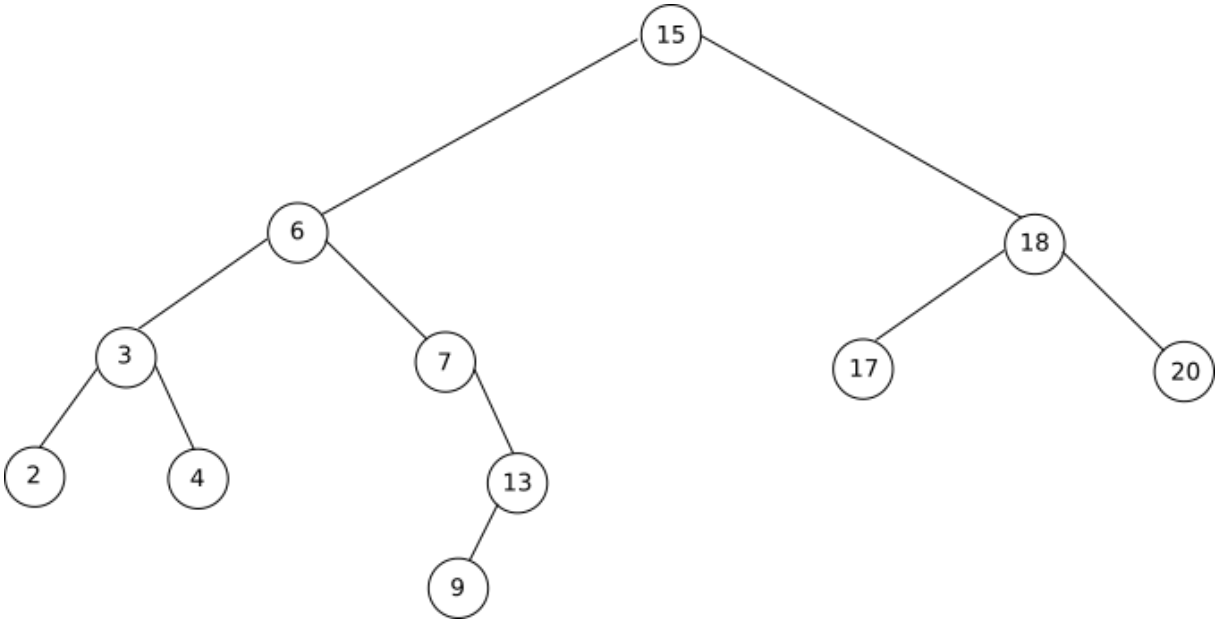
À faire vous-même 10

Appliquez l'algorithme de recherche d'une clé dans un arbre binaire de recherche sur l'arbre ci-dessous. On prendra k = 13.



À faire vous-même 11

Appliquez l'algorithme de recherche d'une clé dans un arbre binaire de recherche sur l'arbre ci-dessous. On prendra k = 16.



Cet algorithme de recherche d'une clé dans un arbre binaire de recherche ressemble beaucoup à la recherche dichotomique vue en première dans le cas où l'arbre binaire de recherche traité est équilibré. La complexité en temps dans le pire des cas de l'algorithme de recherche d'une clé dans un arbre binaire de

recherche équilibré est donc $O(\log 2(n))$. Dans le cas où l'arbre est filiforme, la complexité est $O(n)$. Rappelons qu'un algorithme en $O(\log 2(n))$ est plus "efficace" qu'un algorithme en $O(n)$.
À noter qu'il existe une version dite "itérative" (qui n'est pas récursive) de cet algorithme de recherche :

À faire vous-même 12

Étudiez l'algorithme suivant:

```
VARIABLE
T : arbre
x : noeud
k : entier
DEBUT
ARBRE-RECHERCHE_ITE(T,k) :
  x ← T.racine
  tant que T ≠ NIL et k ≠ x.clé :
    x ← T.racine
    si k < x.clé :
      T ← x.gauche
    sinon :
      T ← x.droit
    fin si
  fin tant que
  si k == x.clé :
    renvoyer vrai
  sinon :
    renvoyer faux
  fin si
FIN
```

Insertion d'une clé dans un arbre binaire de recherche

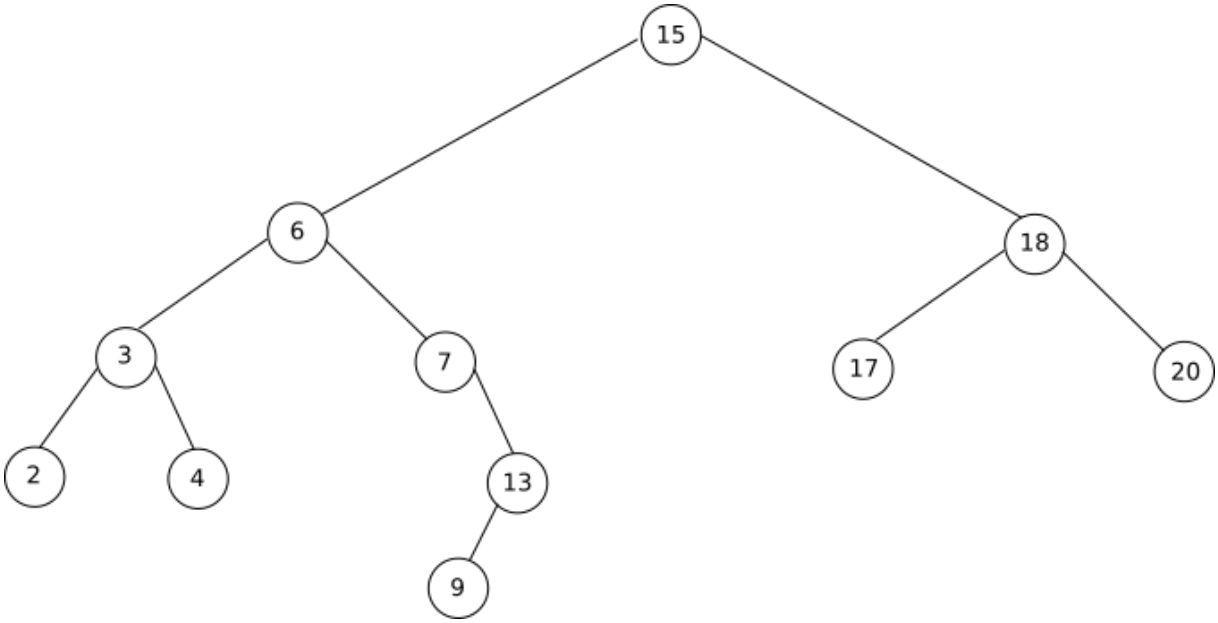
Il est tout à fait possible d'insérer un noeud y dans un arbre binaire de recherche (non vide) :

À faire vous-même 13

Étudiez l'algorithme suivant :

```
VARIABLE
T : arbre
x : noeud
y : noeud
DEBUT
ARBRE-INSERTION(T,y) :
```

```
  x ← T.racine
  tant que T ≠ NIL :
    x ← T.racine
    si y.clé < x.clé :
      T ← x.gauche
    sinon :
      T ← x.droit
    fin si
  fin tant que
  si y.clé < x.clé :
    insérer y à gauche de x
  sinon :
    insérer y à droite de x
  fin si
FIN
```

À faire vous-même 14

Appliquez l'algorithme d'insertion d'un noeud y dans un arbre binaire de recherche sur l'arbre ci-dessous. On prendra y.clé = 16.